

Advanced Level SQL Concepts

Comprehensive Guide with Code Examples

Author: Siddharth Vidyarthi

Table of Contents

1. Introduction to Advanced SQL
2. Window Functions and Analytical Queries
3. Common Table Expressions (CTEs)
4. Recursive CTEs and Hierarchical Data
5. Advanced Subqueries and Correlated Queries
6. Indexing Strategies and Performance Optimization
7. Query Execution Plans and Optimization Techniques
8. Stored Procedures and Functions
9. Triggers and Event-Driven Programming
10. Transaction Management and ACID Properties
11. Concurrency Control and Locking Mechanisms
12. Partitioning Techniques
13. Security Best Practices and SQL Injection Prevention
14. Advanced Data Types and JSON Handling
15. Performance Tuning and Best Practices

Chapter 1: Introduction to Advanced SQL

Advanced SQL encompasses sophisticated database querying techniques that go beyond basic SELECT, INSERT, UPDATE, and DELETE operations. These concepts are essential for database developers, analysts, and administrators who need to work with complex data scenarios and optimize database performance^{[1][7][^13]}.

Advanced SQL concepts include analytical functions, complex joins, subqueries, stored procedures, triggers, and performance optimization techniques. These features enable developers to solve complex business problems efficiently and maintain high-performance database systems^{[2][8][^22]}.

Watermark: Siddharth Vidyarthi

Key Areas of Advanced SQL

Window Functions: Perform calculations across a set of table rows related to the current row without requiring a GROUP BY clause^{[3][15]}.

Common Table Expressions (CTEs): Named temporary result sets that exist within the scope of a single statement^{[17][23][^26]}.

Recursive Queries: Handle hierarchical or tree-structured data efficiently using recursive CTEs^{[20][29]}.

Performance Optimization: Index strategies, query plans, and optimization techniques for high-performance systems^{[19][22][^25]}.

Chapter 2: Window Functions and Analytical Queries

Window functions are powerful SQL features that perform calculations across a set of rows related to the current row. Unlike aggregate functions, window functions do not cause rows to become grouped into a single output row^{[3][6][^15]}.

Basic Syntax

```
-- Basic window function syntax
SELECT
    column1,
    column2,
    WINDOW_FUNCTION() OVER (
        [PARTITION BY partition_expression]
        [ORDER BY order_expression]
        [ROWS/RANGE frame_specification]
    )
FROM table_name;
```

Watermark: Siddharth Vidyarthi

Common Window Functions

ROW_NUMBER(): Assigns unique sequential integers to rows within a partition.

```
-- Example: Ranking employees by salary within each department
SELECT
    employee_id,
    department_id,
    salary,
    ROW_NUMBER() OVER (
        PARTITION BY department_id
        ORDER BY salary DESC
    ) as salary_rank
FROM employees;
```

RANK() and **DENSE_RANK()**: Provide ranking with gaps (RANK) or without gaps (DENSE_RANK).

```
-- Example: Product sales ranking with ties
SELECT
    product_id,
    sales_amount,
    RANK() OVER (ORDER BY sales_amount DESC) as rank_with_gaps,
    DENSE_RANK() OVER (ORDER BY sales_amount DESC) as rank_no_gaps
FROM product_sales;
```

LAG() and **LEAD()**: Access data from previous or next rows.

```
-- Example: Comparing current month sales with previous month
SELECT
    month_year,
    sales_amount,
    LAG(sales_amount, 1) OVER (ORDER BY month_year) as prev_month_sales,
    sales_amount - LAG(sales_amount, 1) OVER (ORDER BY month_year) as growth
FROM monthly_sales;
```

Watermark: Siddharth Vidyarthi

Running Totals and Moving Averages

```
-- Example: Running total and 3-month moving average
SELECT
    transaction_date,
    amount,
    SUM(amount) OVER (
        ORDER BY transaction_date
        ROWS UNBOUNDED PRECEDING
    ) as running_total,
    AVG(amount) OVER (
        ORDER BY transaction_date
        ROWS 2 PRECEDING
    ) as moving_avg_3_months
FROM transactions
ORDER BY transaction_date;
```

Frame Specifications

Window frames define which rows are included in the window calculation:

- **ROWS**: Physical number of rows
- **RANGE**: Logical range of values
- **UNBOUNDED PRECEDING**: From the start of the partition
- **CURRENT ROW**: The current row
- **UNBOUNDED FOLLOWING**: To the end of the partition

Chapter 3: Common Table Expressions (CTEs)

Common Table Expressions (CTEs) provide a way to write more readable and maintainable SQL queries by creating named temporary result sets^{[17][26][^31]}. CTEs are particularly useful for breaking down complex queries into manageable parts.

Basic CTE Syntax

```
WITH cte_name (column1, column2, ...) AS (  
    -- CTE query definition  
    SELECT ...  
    FROM ...  
    WHERE ...  
)  
SELECT ...  
FROM cte_name  
WHERE ...;
```

Simple CTE Example

```
-- Example: Calculate department averages and compare with individual salaries  
WITH DepartmentAvg AS (  
    SELECT  
        department_id,  
        AVG(salary) as avg_salary  
    FROM employees  
    GROUP BY department_id  
)  
SELECT  
    e.employee_id,  
    e.name,  
    e.salary,  
    da.avg_salary,  
    e.salary - da.avg_salary as salary_difference  
FROM employees e  
JOIN DepartmentAvg da ON e.department_id = da.department_id;
```

Multiple CTEs

You can define multiple CTEs in a single query:

```
-- Example: Sales analysis with multiple CTEs  
WITH  
ProductSales AS (  
    SELECT  
        product_id,
```

```

        SUM(quantity * unit_price) as total_sales
    FROM order_items
    GROUP BY product_id
),
TopProducts AS (
    SELECT
        product_id,
        total_sales,
        ROW_NUMBER() OVER (ORDER BY total_sales DESC) as rank
    FROM ProductSales
)
SELECT
    p.product_name,
    tp.total_sales,
    tp.rank
FROM TopProducts tp
JOIN products p ON tp.product_id = p.product_id
WHERE tp.rank <= 10;

```

CTE vs Subqueries

CTEs offer several advantages over traditional subqueries:

- **Readability:** Easier to understand and maintain
- **Reusability:** Can be referenced multiple times
- **Debugging:** Easier to test individual parts

Watermark: Siddharth Vidyarthi

Chapter 4: Recursive CTEs and Hierarchical Data

Recursive CTEs are powerful tools for handling hierarchical data structures such as organizational charts, category trees, or bill-of-materials^{[17][20][23][29]}.

Recursive CTE Structure

```

WITH RECURSIVE cte_name AS (
    -- Anchor member (base case)
    SELECT initial_columns
    FROM base_table
    WHERE base_condition

    UNION ALL

    -- Recursive member
    SELECT recursive_columns
    FROM base_table
    JOIN cte_name ON join_condition
    WHERE recursive_condition
)
SELECT * FROM cte_name;

```

Employee Hierarchy Example

```
-- Example: Display employee hierarchy with levels
WITH RECURSIVE EmployeeHierarchy AS (
  -- Anchor: Find top-level managers (no supervisor)
  SELECT
    employee_id,
    name,
    supervisor_id,
    1 as level,
    CAST(name AS VARCHAR(1000)) as hierarchy_path
  FROM employees
  WHERE supervisor_id IS NULL

  UNION ALL

  -- Recursive: Find subordinates
  SELECT
    e.employee_id,
    e.name,
    e.supervisor_id,
    eh.level + 1,
    CONCAT(eh.hierarchy_path, ' -&gt; ', e.name)
  FROM employees e
  JOIN EmployeeHierarchy eh ON e.supervisor_id = eh.employee_id
  WHERE eh.level < 10 -- Prevent infinite recursion
)
SELECT
  employee_id,
  name,
  level,
  hierarchy_path
FROM EmployeeHierarchy
ORDER BY level, name;
```

Watermark: Siddharth Vidyarthi

Category Tree Traversal

```
-- Example: Product category tree with subcategories
WITH RECURSIVE CategoryTree AS (
  -- Root categories
  SELECT
    category_id,
    category_name,
    parent_category_id,
    0 as depth,
    category_name as full_path
  FROM categories
  WHERE parent_category_id IS NULL

  UNION ALL

  -- Child categories
```

```

SELECT
    c.category_id,
    c.category_name,
    c.parent_category_id,
    ct.depth + 1,
    CONCAT(ct.full_path, ' / ', c.category_name)
FROM categories c
JOIN CategoryTree ct ON c.parent_category_id = ct.category_id
)
SELECT * FROM CategoryTree
ORDER BY full_path;

```

Bill of Materials Example

```

-- Example: Calculate total quantity needed for a product assembly
WITH RECURSIVE BOM AS (
    -- Top-level product
    SELECT
        product_id,
        component_id,
        quantity_needed,
        1 as level
    FROM bill_of_materials
    WHERE product_id = 'PRODUCT_001'

    UNION ALL

    -- Sub-components
    SELECT
        bom.product_id,
        sub.component_id,
        bom.quantity_needed * sub.quantity_needed,
        bom.level + 1
    FROM bill_of_materials sub
    JOIN BOM bom ON sub.product_id = bom.component_id
)
SELECT
    component_id,
    SUM(quantity_needed) as total_quantity
FROM BOM
GROUP BY component_id;

```

Watermark: Siddharth Vidyarthi

Chapter 5: Advanced Subqueries and Correlated Queries

Subqueries are queries nested within other queries, providing powerful ways to filter, calculate, and transform data. Correlated subqueries reference columns from the outer query, making them particularly useful for row-by-row comparisons^{[2][5][^8]}.

Types of Subqueries

Scalar Subqueries: Return a single value

Row Subqueries: Return a single row

Column Subqueries: Return a single column

Table Subqueries: Return a table result

Correlated Subquery Examples

```
-- Example: Find employees earning above department average
SELECT
    employee_id,
    name,
    salary,
    department_id
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.department_id = e1.department_id
);
```

```
-- Example: Find customers with above-average order values
SELECT
    customer_id,
    customer_name
FROM customers c
WHERE (
    SELECT AVG(total_amount)
    FROM orders o
    WHERE o.customer_id = c.customer_id
) > (
    SELECT AVG(total_amount)
    FROM orders
);
```

Watermark: Siddharth Vidyarthi

EXISTS and NOT EXISTS

```
-- Example: Find customers who have placed orders this year
SELECT
    customer_id,
    customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    AND YEAR(o.order_date) = 2025
);
```

```
-- Example: Find products that have never been ordered
SELECT
    product_id,
    product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);
```

Advanced Subquery Patterns

```
-- Example: Top N per group using subquery
SELECT *
FROM (
    SELECT
        employee_id,
        name,
        salary,
        department_id,
        ROW_NUMBER() OVER (
            PARTITION BY department_id
            ORDER BY salary DESC
        ) as rn
    FROM employees
) ranked
WHERE rn <= 3;
```

Watermark: Siddharth Vidyarthi

Chapter 6: Indexing Strategies and Performance Optimization

Database indexing is crucial for query performance optimization. Understanding different index types and when to use them can dramatically improve query execution times^{[19][22][25][28]}.

Types of Indexes

Clustered Index: Physically sorts table data

Non-clustered Index: Contains pointers to data rows

Composite Index: Covers multiple columns

Covering Index: Includes all columns needed for a query

Partial Index: Covers only rows meeting specific conditions

Index Creation Examples

```
-- Create a simple index
CREATE INDEX idx_employee_lastname
ON employees (last_name);

-- Create a composite index
CREATE INDEX idx_order_customer_date
ON orders (customer_id, order_date);

-- Create a covering index
CREATE INDEX idx_product_covering
ON products (category_id)
INCLUDE (product_name, price, description);

-- Create a partial index
CREATE INDEX idx_active_customers
ON customers (customer_id)
WHERE status = 'Active';
```

Watermark: Siddharth Vidyarthi

Index Design Best Practices

Selectivity: Create indexes on columns with high selectivity (many unique values)

```
-- Good: High selectivity
CREATE INDEX idx_employee_id ON employees (employee_id);

-- Poor: Low selectivity
-- CREATE INDEX idx_employee_gender ON employees (gender);
```

Column Order in Composite Indexes: Most selective column first

```
-- Optimal order: specific date first, then broader category
CREATE INDEX idx_sales_date_region
ON sales (sale_date, region_id);
```

Query Optimization Techniques

Use EXPLAIN/EXPLAIN PLAN to analyze query execution:

```
-- PostgreSQL
EXPLAIN ANALYZE
SELECT * FROM employees
WHERE department_id = 5 AND salary > 50000;

-- SQL Server
SET SHOWPLAN_ALL ON;
```

```
SELECT * FROM employees
WHERE department_id = 5 AND salary > 50000;
```

Index Hints (use sparingly):

```
-- SQL Server index hint
SELECT * FROM employees WITH (INDEX(idx_employee_salary))
WHERE salary > 50000;
```

Watermark: Siddharth Vidyarthi

Performance Monitoring

```
-- Find missing indexes (SQL Server)
SELECT
    migs.avg_total_user_cost * (migs.avg_user_impact / 100.0) * (migs.user_seeks + migs.user_updates) AS avg_impact,
    'CREATE INDEX [missing_index_' + CONVERT(varchar, mig.index_group_handle) + '_' + CONVERT(varchar, mig.index_handle) + ']' AS create_index_statement,
    ' ON ' + mid.statement AS statement,
    ' (' + ISNULL(mid.equality_columns, '') + ' ' + CASE WHEN mid.equality_columns IS NOT NULL AND mid.inequality_columns IS NOT NULL THEN 'AND ' + ISNULL(mid.inequality_columns, '') + ' ' + ISNULL(' INCLUDE (' + mid.included_columns + ')', '') AS create_index_statement,
    migs.*,
    mid.database_id,
    mid.object_id
FROM sys.dm_db_missing_index_groups mig
INNER JOIN sys.dm_db_missing_index_group_stats migs ON migs.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details mid ON mig.index_handle = mid.index_handle
WHERE migs.avg_total_user_cost * (migs.avg_user_impact / 100.0) * (migs.user_seeks + migs.user_updates) > 1
ORDER BY migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks + migs.user_updates) DESC
```

Chapter 7: Query Execution Plans and Optimization Techniques

Understanding query execution plans is essential for database performance tuning. Execution plans show how the database engine processes queries and where optimization opportunities exist^{[49][52][^58]}.

Reading Execution Plans

Execution plans display operations as a tree structure:

- **Scan Operations:** Table scan, index scan
- **Seek Operations:** Index seek (more efficient)
- **Join Operations:** Nested loops, hash joins, merge joins
- **Sort Operations:** Explicit sorting operations

```
-- Enable execution plan display (SQL Server)
SET SHOWPLAN_XML ON;

-- Sample query to analyze
SELECT
    c.customer_name,
    COUNT(o.order_id) as order_count,
    SUM(o.total_amount) as total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.registration_date >= '2024-01-01'
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) > 5
ORDER BY total_spent DESC;
```

Watermark: Siddharth Vidyarthi

Query Optimization Strategies

Avoid SELECT *: Specify only needed columns

```
-- Inefficient
SELECT * FROM large_table WHERE condition;

-- Efficient
SELECT id, name, status FROM large_table WHERE condition;
```

Use Appropriate JOIN Types:

```
-- Inner join when you only need matching records
SELECT c.customer_name, o.order_date
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- EXISTS instead of IN for better performance
SELECT customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Optimize WHERE Clauses:

```
-- Use SARGable predicates (Search ARGument able)
-- Good: Can use index
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01';

-- Poor: Cannot use index effectively
WHERE YEAR(order_date) = 2024;
```

Statistics and Cardinality Estimation

Database statistics help the optimizer make better decisions:

```
-- Update statistics (SQL Server)
UPDATE STATISTICS customers;
UPDATE STATISTICS orders;

-- View statistics information
DBCC SHOW_STATISTICS('customers', 'idx_customer_registration_date');
```

Query Hints and Plan Guides

```
-- Force specific join algorithm (SQL Server)
SELECT /*+ USE_HASH(c, o) */
    c.customer_name,
    o.order_date
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;

-- Limit parallelism
SELECT customer_id, SUM(total_amount)
FROM orders
GROUP BY customer_id
OPTION (MAXDOP 4);
```

Chapter 8: Stored Procedures and Functions

Stored procedures and functions encapsulate business logic within the database, improving security, performance, and maintainability^{[18][21][27][44][47]}.

Stored Procedures

Stored procedures are precompiled SQL statements that can accept parameters and return results:

```
-- Create a stored procedure for employee management
CREATE PROCEDURE sp_UpdateEmployeeSalary
    @EmployeeID INT,
    @NewSalary DECIMAL(10,2),
    @EffectiveDate DATE = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Set default effective date if not provided
    IF @EffectiveDate IS NULL
        SET @EffectiveDate = GETDATE();
```

```

-- Validate employee exists
IF NOT EXISTS (SELECT 1 FROM employees WHERE employee_id = @EmployeeID)
BEGIN
    RAISERROR('Employee ID %d does not exist', 16, 1, @EmployeeID);
    RETURN -1;
END

-- Insert salary history record
INSERT INTO salary_history (employee_id, old_salary, new_salary, change_date)
SELECT
    employee_id,
    current_salary,
    @NewSalary,
    @EffectiveDate
FROM employees
WHERE employee_id = @EmployeeID;

-- Update current salary
UPDATE employees
SET current_salary = @NewSalary,
    last_salary_update = @EffectiveDate
WHERE employee_id = @EmployeeID;

-- Return success
RETURN 0;
END;

```

Watermark: Siddharth Vidyarthi

User-Defined Functions

Scalar Functions return single values:

```

-- Create a scalar function to calculate age
CREATE FUNCTION fn_CalculateAge
(
    @BirthDate DATE,
    @AsOfDate DATE = NULL
)
RETURNS INT
AS
BEGIN
    IF @AsOfDate IS NULL
        SET @AsOfDate = GETDATE();

    DECLARE @Age INT;

    SET @Age = DATEDIFF(YEAR, @BirthDate, @AsOfDate);

    -- Adjust for birthday not yet occurred this year
    IF MONTH(@BirthDate) > MONTH(@AsOfDate)
        OR (MONTH(@BirthDate) = MONTH(@AsOfDate) AND DAY(@BirthDate) > DAY(@AsOfDate))
        SET @Age = @Age - 1;

```

```

        RETURN @Age;
    END;

-- Usage
SELECT
    employee_id,
    name,
    birth_date,
    dbo.fn_CalculateAge(birth_date, DEFAULT) as current_age
FROM employees;

```

Table-Valued Functions return tables:

```

-- Create an inline table-valued function
CREATE FUNCTION fn_GetEmployeesByDepartment
(
    @DepartmentID INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        employee_id,
        name,
        salary,
        hire_date
    FROM employees
    WHERE department_id = @DepartmentID
);

-- Usage
SELECT * FROM dbo.fn_GetEmployeesByDepartment(5);

```

Watermark: Siddharth Vidyarthi

Advanced Stored Procedure Features

Error Handling with TRY-CATCH:

```

CREATE PROCEDURE sp_TransferFunds
    @FromAccountID INT,
    @ToAccountID INT,
    @Amount DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Check sufficient funds
        DECLARE @Balance DECIMAL(10,2);

```

```

SELECT @Balance = balance FROM accounts WHERE account_id = @FromAccountID;

IF @Balance < @Amount
BEGIN
    RAISERROR('Insufficient funds', 16, 1);
END

-- Debit from account
UPDATE accounts
SET balance = balance - @Amount
WHERE account_id = @FromAccountID;

-- Credit to account
UPDATE accounts
SET balance = balance + @Amount
WHERE account_id = @ToAccountID;

-- Log transaction
INSERT INTO transaction_log (from_account, to_account, amount, transaction_date)
VALUES (@FromAccountID, @ToAccountID, @Amount, GETDATE());

COMMIT TRANSACTION;

PRINT 'Transfer completed successfully';

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
    DECLARE @ErrorState INT = ERROR_STATE();

    RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);
END CATCH
END;

```

Chapter 9: Triggers and Event-Driven Programming

Triggers are special stored procedures that automatically execute in response to database events. They're essential for maintaining data integrity and implementing business rules^[18][24]^[30][32].

Types of Triggers

DML Triggers: Fire on INSERT, UPDATE, DELETE

DDL Triggers: Fire on schema changes

Logon Triggers: Fire on user logon events

AFTER Triggers

```
-- Create an audit trail trigger
CREATE TRIGGER tr_employee_audit
ON employees
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Handle INSERT
    IF EXISTS (SELECT * FROM inserted) AND NOT EXISTS (SELECT * FROM deleted)
    BEGIN
        INSERT INTO employee_audit (
            employee_id, action_type, action_date, action_user,
            new_salary, new_department_id
        )
        SELECT
            employee_id,
            'INSERT',
            GETDATE(),
            SYSTEM_USER,
            current_salary,
            department_id
        FROM inserted;
    END

    -- Handle UPDATE
    IF EXISTS (SELECT * FROM inserted) AND EXISTS (SELECT * FROM deleted)
    BEGIN
        INSERT INTO employee_audit (
            employee_id, action_type, action_date, action_user,
            old_salary, new_salary, old_department_id, new_department_id
        )
        SELECT
            i.employee_id,
            'UPDATE',
            GETDATE(),
            SYSTEM_USER,
            d.current_salary,
            i.current_salary,
            d.department_id,
            i.department_id
        FROM inserted i
        INNER JOIN deleted d ON i.employee_id = d.employee_id;
    END

    -- Handle DELETE
    IF NOT EXISTS (SELECT * FROM inserted) AND EXISTS (SELECT * FROM deleted)
    BEGIN
        INSERT INTO employee_audit (
            employee_id, action_type, action_date, action_user,
            old_salary, old_department_id
        )
        SELECT
            employee_id,
```

```

        'DELETE',
        GETDATE(),
        SYSTEM_USER,
        current_salary,
        department_id
    FROM deleted;
END
END;

```

Watermark: Siddharth Vidyarthi

INSTEAD OF Triggers

Used primarily with views to make them updatable:

```

-- Create a view for employee summary
CREATE VIEW vw_employee_summary AS
SELECT
    e.employee_id,
    e.name,
    e.current_salary,
    d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id;

-- Create INSTEAD OF trigger for the view
CREATE TRIGGER tr_employee_summary_update
ON vw_employee_summary
INSTEAD OF UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Update employees table
    UPDATE employees
    SET
        name = i.name,
        current_salary = i.current_salary
    FROM employees e
    INNER JOIN inserted i ON e.employee_id = i.employee_id;

    -- Update department if changed
    UPDATE employees
    SET department_id = d.department_id
    FROM employees e
    INNER JOIN inserted i ON e.employee_id = i.employee_id
    INNER JOIN departments d ON i.department_name = d.department_name
    WHERE i.department_name <> d.department_name; (
        SELECT department_name
        FROM departments
        WHERE department_id = e.department_id
    );
END;

```

Business Rule Enforcement

```
-- Trigger to enforce salary constraints
CREATE TRIGGER tr_salary_validation
ON employees
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Check for negative salaries
    IF EXISTS (SELECT * FROM inserted WHERE current_salary < 0)
    BEGIN
        RAISERROR('Salary cannot be negative', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- Check for excessive salary increases
    IF UPDATE(current_salary)
    BEGIN
        IF EXISTS (
            SELECT *
            FROM inserted i
            INNER JOIN deleted d ON i.employee_id = d.employee_id
            WHERE i.current_salary > d.current_salary * 1.5
        )
        BEGIN
            RAISERROR('Salary increase cannot exceed 50%', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END
    END
END;
END;
```

Watermark: Siddharth Vidyarthi

Chapter 10: Transaction Management and ACID Properties

Database transactions ensure data consistency and integrity through the ACID properties: Atomicity, Consistency, Isolation, and Durability^{[33][36][39][42][45]}.

ACID Properties Explained

Atomicity: Transactions are all-or-nothing operations

Consistency: Database remains in a valid state

Isolation: Concurrent transactions don't interfere

Durability: Committed changes are permanent

Transaction Control Statements

```
-- Basic transaction structure
BEGIN TRANSACTION;

    -- Perform multiple related operations
    INSERT INTO orders (customer_id, order_date, total_amount)
    VALUES (123, GETDATE(), 250.00);

    DECLARE @OrderID INT = SCOPE_IDENTITY();

    INSERT INTO order_items (order_id, product_id, quantity, unit_price)
    VALUES
        (@OrderID, 1001, 2, 50.00),
        (@OrderID, 1002, 1, 150.00);

    UPDATE products
    SET stock_quantity = stock_quantity - 2
    WHERE product_id = 1001;

    UPDATE products
    SET stock_quantity = stock_quantity - 1
    WHERE product_id = 1002;

COMMIT TRANSACTION;
```

Watermark: Siddharth Vidyarthi

Isolation Levels

Different isolation levels provide varying degrees of data consistency:

```
-- Set isolation level
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Example demonstrating isolation levels
-- Session 1
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- Don't commit yet

-- Session 2 (different behavior based on isolation level)
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED; -- Can see uncommitted changes
SELECT balance FROM accounts WHERE account_id = 1;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED; -- Waits for commit
SELECT balance FROM accounts WHERE account_id = 1;
```

Deadlock Prevention

```
-- Consistent resource ordering to prevent deadlocks
CREATE PROCEDURE sp_TransferBetweenAccounts
    @Account1 INT,
    @Account2 INT,
    @Amount DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON;

    -- Order accounts consistently to prevent deadlocks
    DECLARE @FirstAccount INT, @SecondAccount INT;

    IF @Account1 < @Account2
    BEGIN
        SET @FirstAccount = @Account1;
        SET @SecondAccount = @Account2;
    END
    ELSE
    BEGIN
        SET @FirstAccount = @Account2;
        SET @SecondAccount = @Account1;
    END

    BEGIN TRANSACTION;

    -- Lock accounts in consistent order
    UPDATE accounts SET balance = balance
    WHERE account_id = @FirstAccount;

    UPDATE accounts SET balance = balance
    WHERE account_id = @SecondAccount;

    -- Perform the actual transfer
    UPDATE accounts SET balance = balance - @Amount
    WHERE account_id = @Account1;

    UPDATE accounts SET balance = balance + @Amount
    WHERE account_id = @Account2;

    COMMIT TRANSACTION;
END;
```

Watermark: Siddharth Vidyarthi

Savepoints

```
-- Using savepoints for partial rollbacks
BEGIN TRANSACTION;

INSERT INTO customers (customer_name, email)
VALUES ('John Doe', 'john@example.com');
```

```

SAVE TRANSACTION sp_after_customer;

BEGIN TRY
    INSERT INTO orders (customer_id, order_date)
    VALUES (SCOPE_IDENTITY(), GETDATE());

    -- Some complex operation that might fail
    EXEC sp_ComplexBusinessLogic;

END TRY
BEGIN CATCH
    -- Rollback only to savepoint, keep customer insert
    ROLLBACK TRANSACTION sp_after_customer;

    -- Log the error but continue
    INSERT INTO error_log (error_message, error_date)
    VALUES (ERROR_MESSAGE(), GETDATE());
END CATCH;

COMMIT TRANSACTION;

```

Chapter 11: Concurrency Control and Locking Mechanisms

Database concurrency control ensures that multiple users can access the database simultaneously while maintaining data integrity^{[9][12]}.

Lock Types

Shared Locks (S): Allow reading, prevent writing

Exclusive Locks (X): Prevent both reading and writing

Update Locks (U): Prevent deadlocks during read-modify-write

Intent Locks (IS, IX, IU): Hierarchy locks for efficiency

Locking Hints

```

-- Force specific lock behavior
SELECT * FROM products WITH (NOLOCK); -- Read uncommitted
SELECT * FROM products WITH (HOLDLOCK); -- Hold shared locks
SELECT * FROM products WITH (UPDLOCK); -- Use update locks
SELECT * FROM products WITH (XLOCK); -- Exclusive locks
SELECT * FROM products WITH (ROWLOCK); -- Row-level locks only
SELECT * FROM products WITH (PAGLOCK); -- Page-level locks
SELECT * FROM products WITH (TABLOCK); -- Table-level locks

```

Optimistic Concurrency Control

```

-- Using timestamps/version numbers for optimistic locking
ALTER TABLE products ADD version_number INT DEFAULT 1;

-- Update with version checking

```

```

UPDATE products
SET
    product_name = @NewName,
    price = @NewPrice,
    version_number = version_number + 1
WHERE
    product_id = @ProductID
    AND version_number = @ExpectedVersion;

-- Check if update was successful
IF @@ROWCOUNT = 0
BEGIN
    RAISERROR('Record was modified by another user', 16, 1);
END

```

Watermark: Siddharth Vidyarthi

Lock Monitoring

```

-- Monitor current locks (SQL Server)
SELECT
    l.resource_type,
    l.resource_database_id,
    l.resource_associated_entity_id,
    l.request_mode,
    l.request_type,
    l.request_status,
    s.session_id,
    s.login_name,
    s.program_name
FROM sys.dm_tran_locks l
LEFT JOIN sys.dm_exec_sessions s ON l.request_session_id = s.session_id
WHERE l.resource_database_id = DB_ID();

-- Find blocking processes
SELECT
    blocking.session_id as blocking_session,
    blocked.session_id as blocked_session,
    blocking_sql.text as blocking_sql,
    blocked_sql.text as blocked_sql
FROM sys.dm_exec_requests blocked
INNER JOIN sys.dm_exec_sessions blocking
    ON blocked.blocking_session_id = blocking.session_id
CROSS APPLY sys.dm_exec_sql_text(blocking.sql_handle) blocking_sql
CROSS APPLY sys.dm_exec_sql_text(blocked.sql_handle) blocked_sql
WHERE blocked.blocking_session_id > 0;

```

Chapter 12: Partitioning Techniques

Table partitioning improves performance and manageability for large databases by dividing tables into smaller, more manageable pieces^{[48][51][57][60][^62]}.

Horizontal Partitioning

```
-- Range partitioning by date (SQL Server)
-- Create partition function
CREATE PARTITION FUNCTION pf_SalesByMonth (DATE)
AS RANGE RIGHT FOR VALUES
('2024-02-01', '2024-03-01', '2024-04-01', '2024-05-01');

-- Create partition scheme
CREATE PARTITION SCHEME ps_SalesByMonth
AS PARTITION pf_SalesByMonth
TO ([PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY]);

-- Create partitioned table
CREATE TABLE sales_partitioned (
    sale_id INT IDENTITY(1,1),
    sale_date DATE,
    customer_id INT,
    amount DECIMAL(10,2),
    INDEX IX_sales_date NONCLUSTERED (sale_date)
) ON ps_SalesByMonth (sale_date);
```

Hash Partitioning

```
-- PostgreSQL hash partitioning example
CREATE TABLE customer_orders (
    order_id SERIAL,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2)
) PARTITION BY HASH (customer_id);

-- Create hash partitions
CREATE TABLE customer_orders_p0 PARTITION OF customer_orders
    FOR VALUES WITH (modulus 4, remainder 0);

CREATE TABLE customer_orders_p1 PARTITION OF customer_orders
    FOR VALUES WITH (modulus 4, remainder 1);

CREATE TABLE customer_orders_p2 PARTITION OF customer_orders
    FOR VALUES WITH (modulus 4, remainder 2);

CREATE TABLE customer_orders_p3 PARTITION OF customer_orders
    FOR VALUES WITH (modulus 4, remainder 3);
```

List Partitioning

```
-- PostgreSQL list partitioning by region
CREATE TABLE sales_by_region (
    sale_id SERIAL,
    region VARCHAR(50),
    sale_date DATE,
    amount DECIMAL(10,2)
) PARTITION BY LIST (region);

CREATE TABLE sales_north PARTITION OF sales_by_region
    FOR VALUES IN ('North', 'Northeast', 'Northwest');

CREATE TABLE sales_south PARTITION OF sales_by_region
    FOR VALUES IN ('South', 'Southeast', 'Southwest');

CREATE TABLE sales_other PARTITION OF sales_by_region
    DEFAULT;
```

Partition Maintenance

```
-- Add new partition (SQL Server)
ALTER PARTITION SCHEME ps_SalesByMonth
NEXT USED [PRIMARY];

ALTER PARTITION FUNCTION pf_SalesByMonth()
SPLIT RANGE ('2024-06-01');

-- Drop old partition
ALTER PARTITION FUNCTION pf_SalesByMonth()
MERGE RANGE ('2024-02-01');

-- Switch partition for maintenance
ALTER TABLE sales_january SWITCH TO sales_partitioned PARTITION 1;
```

Chapter 13: Security Best Practices and SQL Injection Prevention

Database security is critical for protecting sensitive data and preventing unauthorized access^{[50][53][56][59][61][63]}.

SQL Injection Prevention

Parameterized Queries are the primary defense:

```
-- Vulnerable code (DON'T DO THIS)
DECLARE @SQL NVARCHAR(1000);
SET @SQL = 'SELECT * FROM users WHERE username = ''' + @Username + '''';
EXEC sp_executesql @SQL;

-- Secure code (DO THIS)
```

```

DECLARE @SQL NVARCHAR(1000);
SET @SQL = 'SELECT * FROM users WHERE username = @Username';
EXEC sp_executesql @SQL, N'@Username NVARCHAR(50)', @Username;

```

Stored Procedures with Parameters:

```

CREATE PROCEDURE sp_GetUserByCredentials
    @Username NVARCHAR(50),
    @Password NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    -- Hash the password (implement proper hashing)
    DECLARE @HashedPassword NVARCHAR(100);
    SET @HashedPassword = dbo.fn_HashPassword(@Password);

    SELECT
        user_id,
        username,
        email,
        last_login
    FROM users
    WHERE username = @Username
        AND password_hash = @HashedPassword
        AND is_active = 1;
END;

```

Watermark: Siddharth Vidyarthi

Input Validation

```

-- Validate input parameters
CREATE PROCEDURE sp_UpdateUserEmail
    @UserID INT,
    @NewEmail NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate email format
    IF @NewEmail NOT LIKE '%_@_._%'
    BEGIN
        RAISERROR('Invalid email format', 16, 1);
        RETURN;
    END

    -- Validate user exists
    IF NOT EXISTS (SELECT 1 FROM users WHERE user_id = @UserID)
    BEGIN
        RAISERROR('User does not exist', 16, 1);
        RETURN;
    END
END

```

```

-- Update email
UPDATE users
SET email = @NewEmail,
    modified_date = GETDATE()
WHERE user_id = @UserID;
END;

```

Database Roles and Permissions

```

-- Create application-specific roles
CREATE ROLE app_read_only;
CREATE ROLE app_data_entry;
CREATE ROLE app_reporting;

-- Grant appropriate permissions
GRANT SELECT ON customers TO app_read_only;
GRANT SELECT ON products TO app_read_only;

GRANT SELECT, INSERT, UPDATE ON orders TO app_data_entry;
GRANT SELECT, INSERT, UPDATE ON order_items TO app_data_entry;

GRANT SELECT ON sales_summary_view TO app_reporting;
GRANT EXECUTE ON sp_GenerateReport TO app_reporting;

-- Create users and assign roles
CREATE LOGIN app_user1 WITH PASSWORD = 'SecurePassword123!';
CREATE USER app_user1 FOR LOGIN app_user1;
ALTER ROLE app_data_entry ADD MEMBER app_user1;

```

Row-Level Security

```

-- Enable row-level security (SQL Server)
ALTER TABLE customer_data ENABLE ROW LEVEL SECURITY;

-- Create security policy
CREATE FUNCTION fn_SecurityPredicate(@UserRole NVARCHAR(50))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS result
WHERE @UserRole = 'Admin'
    OR (@UserRole = 'Sales' AND SCHEMA_NAME() = 'Sales')
    OR (@UserRole = 'Customer' AND customer_id = CAST(SESSION_CONTEXT(N'CustomerID') AS INT));

-- Apply security policy
CREATE SECURITY POLICY customer_security_policy
ADD FILTER PREDICATE dbo.fn_SecurityPredicate(USER_NAME()) ON customer_data;

```

Chapter 14: Advanced Data Types and JSON Handling

Modern databases support complex data types including JSON, XML, and spatial data, enabling more sophisticated applications^[35].

JSON Data Handling

```
-- SQL Server JSON functions
DECLARE @json NVARCHAR(MAX) = '{
  "customer": {
    "id": 123,
    "name": "John Doe",
    "orders": [
      {"id": 1001, "total": 250.00},
      {"id": 1002, "total": 175.50}
    ]
  }
}';

-- Extract JSON values
SELECT
  JSON_VALUE(@json, '$.customer.id') as customer_id,
  JSON_VALUE(@json, '$.customer.name') as customer_name,
  JSON_QUERY(@json, '$.customer.orders') as orders_json;

-- Query JSON arrays
SELECT
  order_id,
  order_total
FROM OPENJSON(@json, '$.customer.orders')
WITH (
  order_id INT '$.id',
  order_total DECIMAL(10,2) '$.total'
);
```

PostgreSQL JSON Operations:

```
-- Create table with JSON column
CREATE TABLE user_preferences (
  user_id INT PRIMARY KEY,
  preferences JSONB
);

-- Insert JSON data
INSERT INTO user_preferences VALUES
(1, '{"theme": "dark", "language": "en", "notifications": {"email": true, "sms": false}}')
(2, '{"theme": "light", "language": "es", "notifications": {"email": false, "sms": true}}');

-- Query JSON data
SELECT
  user_id,
  preferences->'theme' as theme,
  preferences->'notifications'->'email' as email_notifications
FROM user_preferences;
```

```
-- JSON path queries
SELECT * FROM user_preferences
WHERE preferences @> '{ "theme": "dark" }';
```

Watermark: Siddharth Vidyarthi

XML Data Processing

```
-- SQL Server XML operations
DECLARE @xml XML = '
<customers>
  <customer id="1">
    <name>John Doe</name>
    <email>john@example.com</email>
    <orders>
      <order id="1001" total="250.00"/>
      <order id="1002" total="175.50"/>
    </orders>
  </customer>
</customers>';

-- Extract XML data
SELECT
  c.value('@id', 'INT') as customer_id,
  c.value('name[^1]', 'NVARCHAR(50)') as customer_name,
  c.value('email[^1]', 'NVARCHAR(100)') as email
FROM @xml.nodes('/customers/customer') as t(c);

-- Query XML with XPath
SELECT
  o.value('@id', 'INT') as order_id,
  o.value('@total', 'DECIMAL(10,2)') as total
FROM @xml.nodes('//order') as t(o);
```

Spatial Data Types

```
-- SQL Server spatial data
DECLARE @point GEOGRAPHY = GEOGRAPHY::Point(47.6062, -122.3321, 4326); -- Seattle
DECLARE @polygon GEOGRAPHY = GEOGRAPHY::STPolyFromText('POLYGON((-122.4 47.5, -122.2 47.5, -122.2 47.5, -122.4 47.5, -122.4 47.5))', 4326);

-- Spatial queries
SELECT
  location_name,
  location_point.STDistance(@point) as distance_meters
FROM locations
WHERE location_point.STWithin(@polygon) = 1
ORDER BY location_point.STDistance(@point);
```

Watermark: Siddharth Vidyarthi

Chapter 15: Performance Tuning and Best Practices

Database performance tuning involves multiple strategies to optimize query execution, resource utilization, and overall system performance^{[19][22][28][55]}.

Query Performance Analysis

```
-- Performance monitoring queries (SQL Server)

-- Find expensive queries
SELECT TOP 10
    (qs.total_elapsed_time / qs.execution_count) as avg_elapsed_time,
    qs.execution_count,
    SUBSTRING(st.text, (qs.statement_start_offset/2) + 1,
        ((CASE WHEN qs.statement_end_offset = -1
            THEN LEN(CONVERT(NVARCHAR(MAX), st.text)) * 2
            ELSE qs.statement_end_offset
            END - qs.statement_start_offset)/2) + 1) as query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY avg_elapsed_time DESC;

-- Find queries with high CPU usage
SELECT TOP 10
    (qs.total_worker_time / qs.execution_count) as avg_cpu_time,
    qs.execution_count,
    qs.total_worker_time,
    SUBSTRING(st.text, (qs.statement_start_offset/2) + 1,
        ((CASE WHEN qs.statement_end_offset = -1
            THEN LEN(CONVERT(NVARCHAR(MAX), st.text)) * 2
            ELSE qs.statement_end_offset
            END - qs.statement_start_offset)/2) + 1) as query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY avg_cpu_time DESC;
```

Index Optimization

```
-- Find unused indexes
SELECT
    i.name as index_name,
    OBJECT_NAME(i.object_id) as table_name,
    i.type_desc,
    us.user_seeks,
    us.user_scans,
    us.user_lookups,
    us.user_updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us
    ON i.object_id = us.object_id AND i.index_id = us.index_id
WHERE OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1
    AND i.index_id > 0
    AND (us.user_seeks + us.user_scans + us.user_lookups) < us.user_updates
```

```

ORDER BY (us.user_seeks + us.user_scans + us.user_lookups) ASC;

-- Index fragmentation analysis
SELECT
    OBJECT_NAME(ips.object_id) as table_name,
    i.name as index_name,
    ips.avg_fragmentation_in_percent,
    ips.page_count
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
INNER JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 30
    AND ips.page_count > 1000
ORDER BY ips.avg_fragmentation_in_percent DESC;

```

Watermark: Siddharth Vidyarthi

Memory and Resource Optimization

```

-- Buffer pool analysis
SELECT
    CASE
        WHEN database_id = 32767 THEN 'ResourceDb'
        ELSE DB_NAME(database_id)
    END as database_name,
    COUNT(*) * 8 / 1024 as buffer_pool_mb
FROM sys.dm_os_buffer_descriptors
GROUP BY database_id
ORDER BY buffer_pool_mb DESC;

-- Wait statistics analysis
SELECT
    wait_type,
    wait_time_ms,
    waiting_tasks_count,
    wait_time_ms / waiting_tasks_count as avg_wait_time_ms
FROM sys.dm_os_wait_stats
WHERE waiting_tasks_count > 0
    AND wait_time_ms > 1000
ORDER BY wait_time_ms DESC;

```

Database Maintenance

```

-- Update statistics maintenance
EXEC sp_MSforeachtable 'UPDATE STATISTICS ? WITH FULLSCAN';

-- Index maintenance based on fragmentation
DECLARE @sql NVARCHAR(MAX) = '';
SELECT @sql = @sql +
    CASE
        WHEN avg_fragmentation_in_percent > 30 THEN
            'ALTER INDEX [' + i.name + '] ON [' + OBJECT_NAME(ips.object_id) + '] REBUILD'
        WHEN avg_fragmentation_in_percent > 10 THEN
            'ALTER INDEX [' + i.name + '] ON [' + OBJECT_NAME(ips.object_id) + '] REORGANIZE'
    END

```

```
        ELSE ''
    END
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
INNER JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE i.index_id > 0 AND ips.page_count > 100;

EXEC sp_executesql @sql;
```

Watermark: Siddharth Vidyarthi

Best Practices Summary

Query Writing Best Practices:

- Use appropriate indexes and understand execution plans
- Avoid SELECT * and retrieve only necessary columns
- Use proper JOIN types and conditions
- Implement proper WHERE clause filtering
- Consider using EXISTS instead of IN for subqueries

Database Design Best Practices:

- Normalize data appropriately (usually 3NF)
- Use proper data types and constraints
- Implement proper indexing strategy
- Plan for partitioning on large tables
- Design for scalability and maintainability

Security Best Practices:

- Always use parameterized queries
- Implement proper authentication and authorization
- Use database roles and principle of least privilege
- Regularly audit database access and permissions
- Encrypt sensitive data at rest and in transit

Performance Best Practices:

- Regular maintenance: update statistics, rebuild/reorganize indexes
- Monitor and tune expensive queries
- Implement proper backup and recovery strategies
- Plan for disaster recovery and business continuity
- Use appropriate isolation levels for transactions

Watermark: Siddharth Vidyarthi

Conclusion

This comprehensive guide has covered advanced SQL concepts essential for database professionals. From window functions and CTEs to performance optimization and security, these concepts form the foundation for building robust, scalable database applications.

The key to mastering advanced SQL is practice and continuous learning. Database systems evolve constantly, and staying current with new features and best practices is crucial for success in database development and administration.

Remember that while these advanced features are powerful, they should be used judiciously. Always consider the specific requirements of your application, the characteristics of your data, and the performance implications of your choices.

Watermark: Siddharth Vidyarthi

Key Takeaways:

1. Window functions provide powerful analytical capabilities without grouping data
2. CTEs improve query readability and enable recursive operations
3. Proper indexing is crucial for query performance
4. Understanding execution plans helps optimize query performance
5. Security should be built into every aspect of database design
6. Regular maintenance and monitoring are essential for optimal performance
7. Advanced features should be used appropriately based on specific requirements

Final Note: This guide provides a solid foundation in advanced SQL concepts. Continue to practice these techniques, experiment with different scenarios, and stay updated with the latest developments in database technology to become proficient in advanced SQL programming.

Watermark: Siddharth Vidyarthi

About the Author: Siddharth Vidyarthi is a database professional with expertise in advanced SQL concepts, performance optimization, and database architecture design.

End of Document

Watermark: Siddharth Vidyarthi

